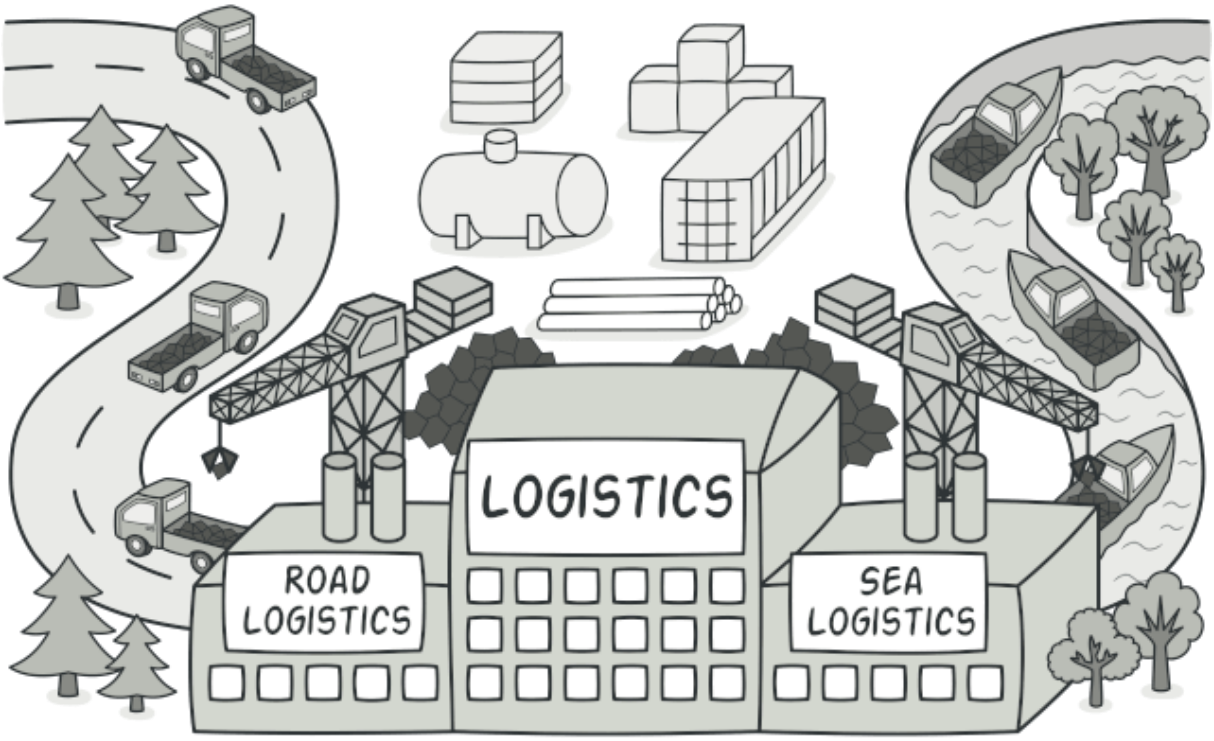
Factory Method

Also known as: Virtual Constructor

Absicht

Factory Method ist ein Erstellungsentwurfsmuster, das eine Schnittstelle zum Erstellen von Objekten in einer Superklasse bereitstellt, Unterklassen jedoch erlaubt, den Typ der zu erstellenden Objekte zu ändern.



Problem

Stellen Sie sich vor, Sie erstellen eine Anwendung für das Logistikmanagement. Die erste Version Ihrer App kann nur den Transport per LKW abwickeln, daher befindet sich der Großteil Ihres Codes innerhalb der `Truck`Klasse.

Nach einer Weile erfreut sich Ihre App großer Beliebtheit. Täglich erhalten Sie Dutzende Anfragen von Seetransportunternehmen, Seelogistik in die App zu integrieren.



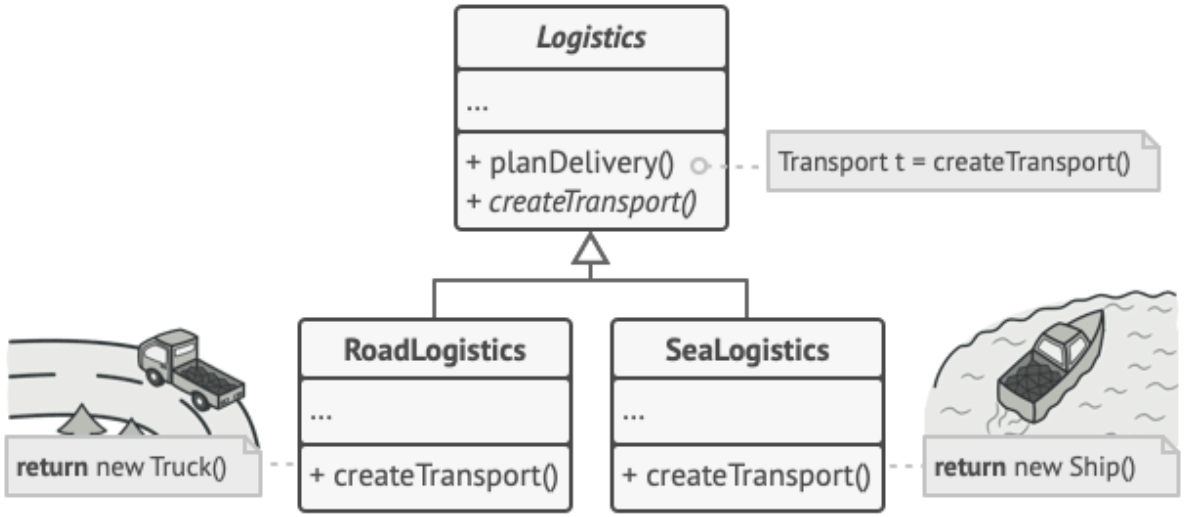
Das Hinzufügen einer neuen Klasse zum Programm ist nicht so einfach, wenn der restliche Code bereits an vorhandene Klassen gekoppelt ist.

Tolle Neuigkeiten, oder? Aber wie sieht es mit dem Code aus? Derzeit ist der Großteil Ihres Codes an die `Truck`Klasse gekoppelt. Das Hinzufügen `Ships`zur App würde Änderungen an der gesamten Codebasis erfordern. Wenn Sie später ein weiteres Verkehrsmittel zur App hinzufügen möchten, müssen Sie wahrscheinlich alle Änderungen erneut vornehmen.

Als Ergebnis erhalten Sie einen ziemlich unangenehmen Code voller Bedingungen, die das Verhalten der App je nach Klasse der Transportobjekte ändern.

Lösung

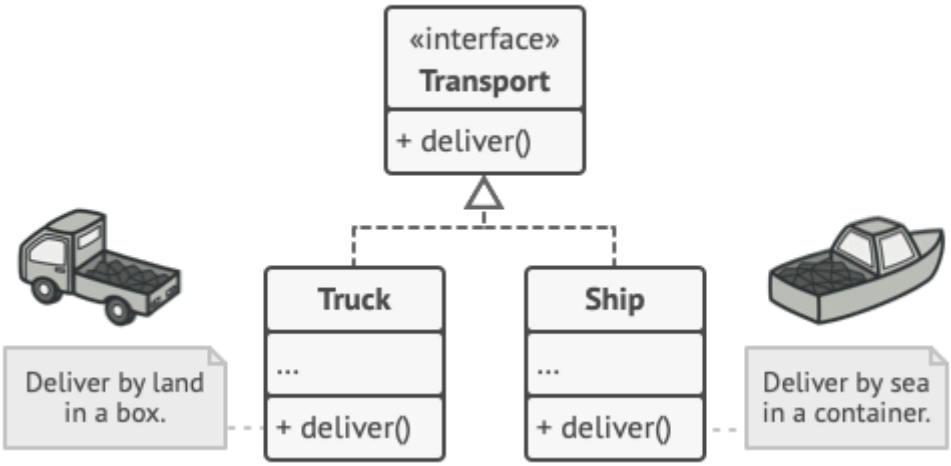
Das Factory-Method-Muster schlägt vor, direkte Objektkonstruktionsaufrufe (mit dem `new`Operator) durch Aufrufe einer speziellen *Factory*- Methode zu ersetzen. Keine Sorge: Die Objekte werden weiterhin über den `new`Operator erstellt, dieser wird jedoch innerhalb der Factory-Methode aufgerufen. Von einer Factory-Methode zurückgegebene Objekte werden oft als Produkte bezeichnet .



Unterklassen können die Klasse der Objekte ändern, die von der Factory-Methode zurückgegeben werden.

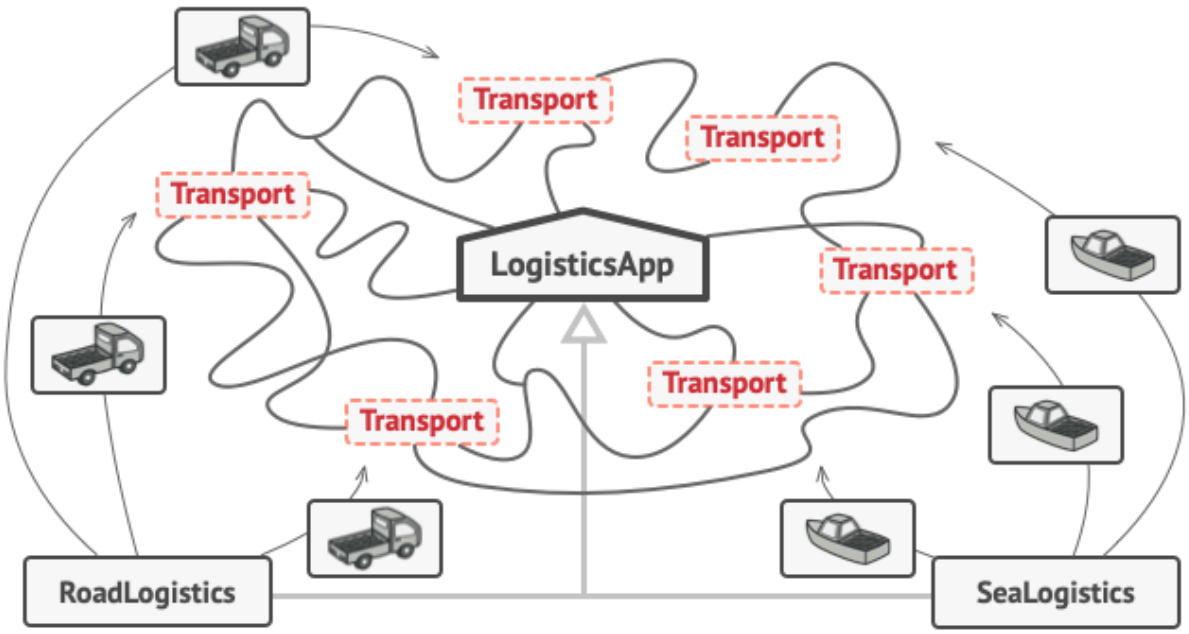
Auf den ersten Blick mag diese Änderung sinnlos erscheinen: Wir haben lediglich den Konstruktoraufruf von einem Programmteil in einen anderen verschoben. Bedenken Sie jedoch Folgendes: Jetzt können Sie die Factory-Methode in einer Unterklasse überschreiben und die von der Methode erstellte Produktklasse ändern.

Es gibt jedoch eine kleine Einschränkung: Unterklassen dürfen nur dann unterschiedliche Produkttypen zurückgeben, wenn diese eine gemeinsame Basisklasse oder Schnittstelle haben. Außerdem sollte der Rückgabetyp der Factory-Methode in der Basisklasse als diese Schnittstelle deklariert sein.



Alle Produkte müssen derselben Schnittstelle folgen.

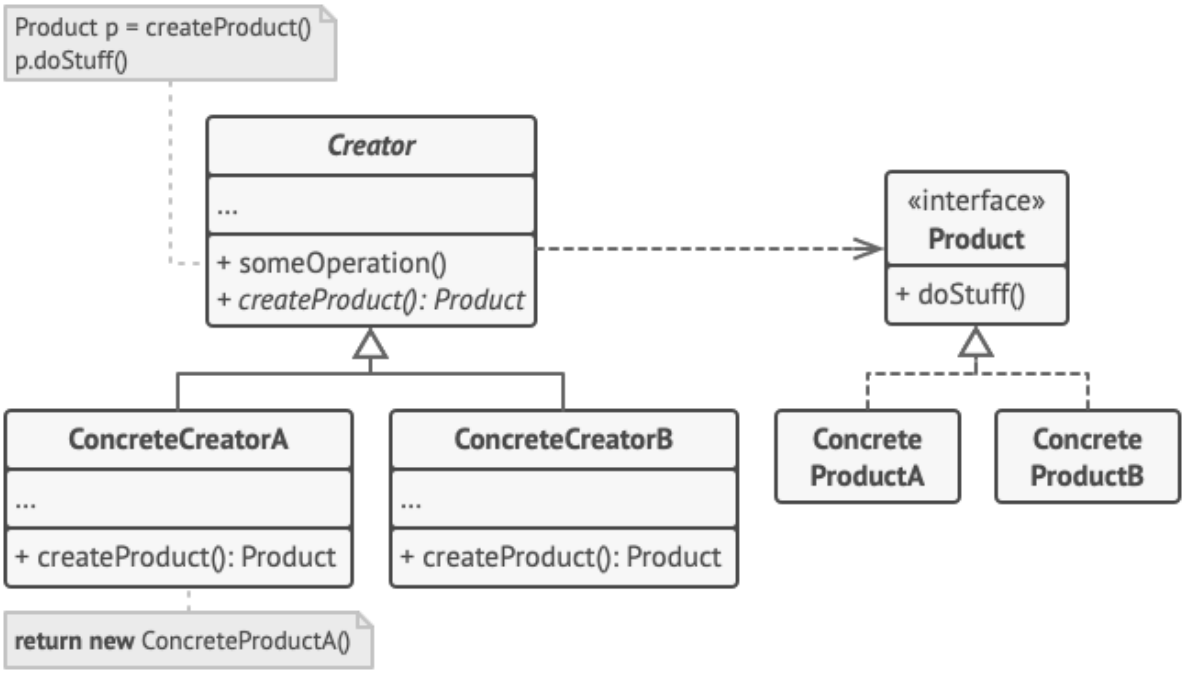
Beispielsweise sollten sowohl die Klassen `Truck`als auch die Schnittstelle implementieren, die eine Methode namens deklariert . Jede Klasse implementiert diese Methode anders: Lkw liefern Fracht auf dem Landweg, Schiffe liefern Fracht auf dem Seeweg. Die Factory-Methode in der Klasse gibt Lkw-Objekte zurück, während die Factory-Methode in der Klasse Schiffe zurückgibt.`ShipTransportdeliverRoadLogisticsSeaLogistics`



Solange alle Produktklassen eine gemeinsame Schnittstelle implementieren, können Sie ihre Objekte an den Clientcode übergeben, ohne ihn zu beschädigen.

Der Code, der die Factory-Methode verwendet (oft als *Client*- Code bezeichnet), erkennt keinen Unterschied zwischen den tatsächlichen Produkten, die von verschiedenen Unterklassen zurückgegeben werden. Der Client behandelt alle Produkte als abstrakt `Transport`. Er weiß, dass alle Transportobjekte die Methode haben sollen `deliver`, aber die genaue Funktionsweise ist für ihn unwichtig.

Struktur



1. Das **Produkt** deklariert die Schnittstelle, die allen Objekten gemeinsam ist, die vom Ersteller und seinen Unterklassen erstellt werden können.
2. **Konkrete Produkte** sind unterschiedliche Implementierungen der Produktschnittstelle.

3. Die **Creator-** Klasse deklariert die Factory-Methode, die neue Produktobjekte zurückgibt. Es ist wichtig, dass der Rückgabetyp dieser Methode mit der Produktschnittstelle übereinstimmt.

   Sie können die Factory-Methode deklarieren, um `abstract`alle Unterklassen zu zwingen, ihre eigenen Versionen der Methode zu implementieren. Alternativ kann die Basis-Factory-Methode einen Standardprodukttyp zurückgeben.

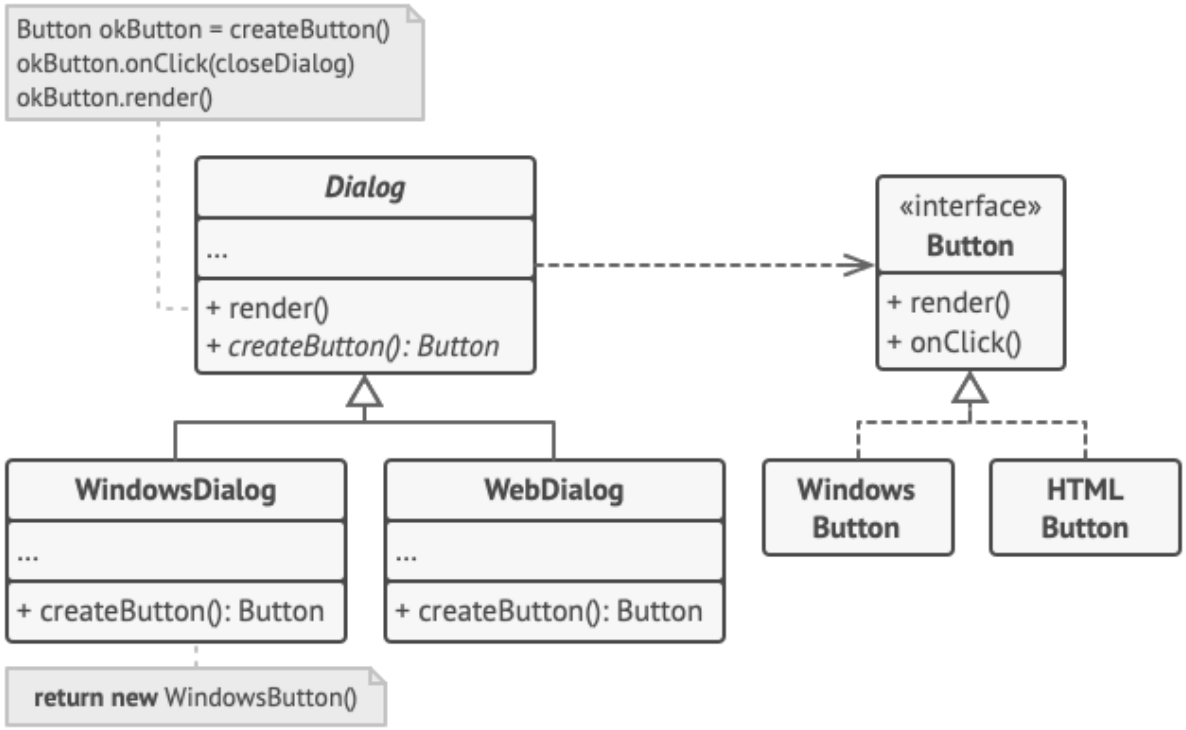
   Beachten Sie, dass die Produkterstellung trotz des Namens **nicht** die Hauptverantwortung des Erstellers ist. Normalerweise verfügt die Erstellerklasse bereits über eine produktbezogene Kerngeschäftslogik. Die Factory-Methode hilft, diese Logik von den konkreten Produktklassen zu entkoppeln. Ein Vergleich: Ein großes Softwareentwicklungsunternehmen kann eine Schulungsabteilung für Programmierer haben. Die Hauptaufgabe des Unternehmens als Ganzes besteht jedoch weiterhin darin, Code zu schreiben, nicht darin, Programmierer auszubilden.

4. **Konkrete Ersteller** überschreiben die grundlegende Factory-Methode, sodass sie einen anderen Produkttyp zurückgeben.

   Beachten Sie, dass die Factory-Methode nicht ständig neue Instanzen **erstellen muss. Sie kann auch vorhandene Objekte aus einem Cache, einem Objektpool oder einer anderen Quelle zurückgeben.**

Pseudocode

Dieses Beispiel veranschaulicht, wie die **Factory-Methode** zum Erstellen plattformübergreifender UI-Elemente verwendet werden kann, ohne den Client-Code an konkrete UI-Klassen zu koppeln.



Das plattformübergreifende Dialogbeispiel.

Die Basisklasse `Dialog` verwendet verschiedene UI-Elemente zum Rendern ihres Fensters. Unter verschiedenen Betriebssystemen können diese Elemente etwas anders aussehen, ihr Verhalten sollte aber dennoch konsistent sein. Eine Schaltfläche unter Windows ist auch unter Linux eine Schaltfläche.

Mit der Factory-Methode müssen Sie die Klassenlogik nicht `Dialog` für jedes Betriebssystem neu schreiben. Deklarieren wir eine Factory-Methode, die Schaltflächen innerhalb der Basisklasse erzeugt `Dialog`, können wir später eine Unterklasse erstellen, die aus der Factory-Methode Schaltflächen im Windows-Stil zurückgibt. Die Unterklasse übernimmt dann den Großteil des Codes von der Basisklasse, kann aber dank der Factory-Methode Schaltflächen im Windows-Stil auf dem Bildschirm darstellen.

Damit dieses Muster funktioniert, `Dialog`muss die Basisklasse mit abstrakten Schaltflächen arbeiten: einer Basisklasse oder einer Schnittstelle, der alle konkreten Schaltflächen folgen. Auf diese Weise `Dialog`bleibt der Code funktionsfähig, unabhängig von der Art der Schaltflächen, mit denen er arbeitet.

Natürlich können Sie diesen Ansatz auch auf andere UI-Elemente anwenden. Mit jeder neuen Factory-Methode, die Sie hinzufügen `Dialog`, nähern Sie sich jedoch dem **Abstract-Factory** -Muster an. Keine Sorge, wir sprechen später darüber.

```
// The creator class declares the factory method that must
// return an object of a product class. The creator's subclasses
// usually provide the implementation of this method.
class Dialog is
    // The creator may also provide some default implementation
    // of the factory method.
    abstract method createButton():Button

    // Note that, despite its name, the creator's primary
    // responsibility isn't creating products. It usually
    // contains some core business logic that relies on product
    // objects returned by the factory method. Subclasses can
    // indirectly change that business logic by overriding the
    // factory method and returning a different type of product
    // from it.
    method render() is
        // Call the factory method to create a product object.
        Button okButton = createButton()
        // Now use the product.
        okButton.onClick(closeDialog)
        okButton.render()
```

```
// Concrete creators override the factory method to change the
// resulting product's type.
class WindowsDialog extends Dialog is
    method createButton():Button is
        return new WindowsButton()

class WebDialog extends Dialog is
    method createButton():Button is
        return new HTMLButton()


// The product interface declares the operations that all
// concrete products must implement.
interface Button is
    method render()
    method onClick(f)

// Concrete products provide various implementations of the
// product interface.
class WindowsButton implements Button is
    method render(a, b) is
        // Render a button in Windows style.
    method onClick(f) is
        // Bind a native OS click event.

class HTMLButton implements Button is
    method render(a, b) is
        // Return an HTML representation of a button.
    method onClick(f) is
        // Bind a web browser click event.


class Application is
    field dialog: Dialog

    // The application picks a creator's type depending on the
    // current configuration or environment settings.
    method initialize() is
        config = readApplicationConfigFile()

        if (config.OS == "Windows") then
            dialog = new WindowsDialog()
        else if (config.OS == "Web") then
            dialog = new WebDialog()
        else
            throw new Exception("Error! Unknown operating system.")

    // The client code works with an instance of a concrete
    // creator, albeit through its base interface. As long as
    // the client keeps working with the creator via the base
    // interface, you can pass it any creator's subclass.
    method main() is
        this.initialize()
        dialog.render()
```

Anwendbarkeit

Verwenden Sie die Factory-Methode, wenn Sie die genauen Typen und Abhängigkeiten der Objekte, mit denen Ihr Code arbeiten soll, nicht im Voraus kennen.
Die Factory-Methode trennt den Produktkonstruktionscode vom Code, der das Produkt tatsächlich verwendet. Daher ist es einfacher, den Produktkonstruktionscode unabhängig vom Rest des Codes zu erweitern.

Um beispielsweise der App einen neuen Produkttyp hinzuzufügen, müssen Sie nur eine neue Creator-Unterklasse erstellen und die Factory-Methode darin überschreiben.

Verwenden Sie die Factory-Methode, wenn Sie den Benutzern Ihrer Bibliothek oder Ihres Frameworks eine Möglichkeit bieten möchten, die internen Komponenten zu erweitern.
Vererbung ist wahrscheinlich der einfachste Weg, das Standardverhalten einer Bibliothek oder eines Frameworks zu erweitern. Doch wie erkennt das Framework, dass Ihre Unterklasse anstelle einer Standardkomponente verwendet werden soll?

Die Lösung besteht darin, den Code, der Komponenten im gesamten Framework erstellt, auf eine einzige Factory-Methode zu reduzieren und jedem zu ermöglichen, diese Methode zu überschreiben und die Komponente selbst zu erweitern.

Sehen wir uns an, wie das funktionieren würde. Stellen Sie sich vor, Sie schreiben eine App mit einem Open-Source-UI-Framework. Ihre App sollte runde Schaltflächen haben, das Framework bietet jedoch nur eckige. Sie erweitern die Standardklasse `Button`um eine großartige `RoundButton`Unterklasse. Nun müssen Sie der Hauptklasse jedoch mitteilen, `UIFramework`dass sie die neue Schaltflächenunterklasse anstelle der Standardklasse verwenden soll.

Dazu erstellen Sie eine Unterklasse `UIWithRoundButtons`einer Basis-Frameworkklasse und überschreiben deren `createButton`Methode. Während diese Methode `Button`Objekte der Basisklasse zurückgibt, sorgen Sie dafür, dass Ihre Unterklasse `RoundButton`Objekte zurückgibt. Verwenden Sie nun die `UIWithRoundButtons`Klasse anstelle von `UIFramework`. Und das war’s schon!

Verwenden Sie die Factory-Methode, wenn Sie Systemressourcen sparen möchten, indem Sie vorhandene Objekte wiederverwenden, anstatt sie jedes Mal neu zu erstellen.
Dieser Bedarf besteht häufig beim Umgang mit großen, ressourcenintensiven Objekten wie Datenbankverbindungen, Dateisystemen und Netzwerkressourcen.

Überlegen wir, was getan werden muss, um ein vorhandenes Objekt wiederzuverwenden:

1. Zunächst müssen Sie Speicherplatz erstellen, um den Überblick über alle erstellten Objekte zu behalten.
2. Wenn jemand ein Objekt anfordert, sollte das Programm in diesem Pool nach einem freien Objekt suchen.
3. … und geben Sie es dann an den Clientcode zurück.

4. Wenn keine freien Objekte vorhanden sind, sollte das Programm ein neues erstellen (und es dem Pool hinzufügen).

Das ist eine Menge Code! Und er muss an einem einzigen Ort abgelegt werden, damit das Programm nicht mit doppeltem Code verunreinigt wird.

Der naheliegendste und praktischste Ort für diesen Code ist wahrscheinlich der Konstruktor der Klasse, deren Objekte wir wiederverwenden möchten. Ein Konstruktor muss jedoch per Definition immer **neue Objekte** zurückgeben . Er kann keine vorhandenen Instanzen zurückgeben.

Daher benötigen Sie eine reguläre Methode, die sowohl neue Objekte erstellen als auch vorhandene wiederverwenden kann. Das klingt sehr nach einer Factory-Methode.

So implementieren Sie

1. Sorgen Sie dafür, dass alle Produkte die gleiche Schnittstelle verwenden. Diese Schnittstelle sollte Methoden deklarieren, die für jedes Produkt sinnvoll sind.
2. Fügen Sie innerhalb der Creator-Klasse eine leere Factory-Methode hinzu. Der Rückgabetyp der Methode sollte der gemeinsamen Produktschnittstelle entsprechen.
3. Suchen Sie im Code des Erstellers nach allen Verweisen auf Produktkonstruktoren. Ersetzen Sie diese nacheinander durch Aufrufe der Factory-Methode, während Sie den Code zur Produkterstellung in die Factory-Methode extrahieren.

   Möglicherweise müssen Sie der Factory-Methode einen temporären Parameter hinzufügen, um den Typ des zurückgegebenen Produkts zu steuern.

   An dieser Stelle sieht der Code der Factory-Methode möglicherweise ziemlich unschön aus. Er enthält möglicherweise eine umfangreiche `switch`Anweisung, die die zu instanziierende Produktklasse auswählt. Aber keine Sorge, wir werden das Problem bald beheben.

4. Erstellen Sie nun für jeden in der Factory-Methode aufgeführten Produkttyp eine Reihe von Creator-Unterklassen. Überschreiben Sie die Factory-Methode in den Unterklassen und extrahieren Sie die entsprechenden Konstruktionscodes aus der Basismethode.
5. Wenn es zu viele Produkttypen gibt und es keinen Sinn ergibt, für alle Unterklassen zu erstellen, können Sie den Steuerparameter aus der Basisklasse in Unterklassen wiederverwenden.

   Stellen Sie sich beispielsweise folgende Klassenhierarchie vor: die Basisklasse `Mail`mit einigen Unterklassen: `AirMail`und `GroundMail`; die `Transport`Klassen sind `Plane`, `Truck`und `Train`. Obwohl die `AirMail`Klasse nur `Plane`Objekte verwendet, kann sie sowohl mit als auch mit Objekten `GroundMail`arbeiten . Sie können eine neue Unterklasse (z. B.)

erstellen, um beide Fälle zu behandeln, aber es gibt noch eine andere Möglichkeit. Der Clientcode kann ein Argument an die Factory-Methode der Klasse übergeben, um zu steuern, welches Produkt er erhalten möchte.`TruckTrainTrainMailGroundMail`

6. Wenn die Basis-Factory-Methode nach allen Extraktionen leer ist, können Sie sie abstrakt machen. Wenn noch etwas übrig ist, können Sie es zum Standardverhalten der Methode machen.

Für und Wider

- Sie vermeiden eine enge Kopplung zwischen dem Ersteller und den konkreten Produkten.
- *Single-Responsibility-Prinzip* . Sie können den Code zur Produkterstellung an eine Stelle im Programm verschieben, wodurch der Code leichter zu unterstützen ist.
- *Offenes/Geschlossenes Prinzip* . Sie können neue Produkttypen in das Programm einführen, ohne den vorhandenen Clientcode zu beschädigen.

- Der Code kann komplizierter werden, da Sie zur Implementierung des Musters viele neue Unterklassen einführen müssen. Im Idealfall führen Sie das Muster in eine bestehende Hierarchie von Erstellerklassen ein.

Beziehungen zu anderen Mustern

- Viele Designs beginnen mit der Verwendung **der Factory-Methode** (weniger kompliziert und über Unterklassen besser anpassbar) und entwickeln sich in Richtung **Abstract Factory** , **Prototype** oder **Builder** (flexibler, aber komplizierter).
- **Abstrakte Factory** -Klassen basieren häufig auf einer Reihe von **Factory-Methoden** , Sie können jedoch auch **Prototype** verwenden , um die Methoden dieser Klassen zusammenzustellen.
- Sie können **Factory Method** zusammen mit **Iterator** verwenden , damit Sammlungsunterklassen verschiedene Typen von Iteratoren zurückgeben, die mit den Sammlungen kompatibel sind.
- **Prototype** basiert nicht auf Vererbung und hat daher keine Nachteile. Andererseits erfordert *Prototype* eine komplizierte Initialisierung des geklonten Objekts. **Factory Method** basiert auf Vererbung, erfordert aber keinen Initialisierungsschritt.
- **Die Factory-Methode** ist eine Spezialisierung der **Template-Methode** . Gleichzeitig kann eine *Factory-Methode* als Schritt in einer großen *Template-Methode* dienen .